

LUIZA MANCHINI

The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, with the 'biblioteca' database selected. The main pane shows a SQL query being executed. The query is a PL/pgSQL function that updates the number of pages for a book and raises an exception if the book is not found. The results pane shows the output of the query, which is a table with 4 rows.

```
Query History
-- Numero de paginas invalido. Informe um valor maior que 10.;
END IF;
-- Atualiza o livro
UPDATE livro
SET num_paginas = p_paginas
WHERE id_livro = p_id_livro;
-- Verifica se o livro foi encontrado
IF NOT FOUND THEN
RAISE EXCEPTION
'livro de ID % não foi encontrado.',
p_id_livro;
END IF;
RAISE NOTICE
'livro de ID % atualizado para % páginas.',
p_id_livro,
p_paginas;
END;
$$;
SELECT id_livro, titulo, num_paginas
FROM livro
ORDER BY id_livro;
```

id_livro	titulo	num_paginas
1	O Senhor dos Anéis	1568
2	Dom Casmurro	208
3	A Hora da Estrela	88
4	O Hobbit	336

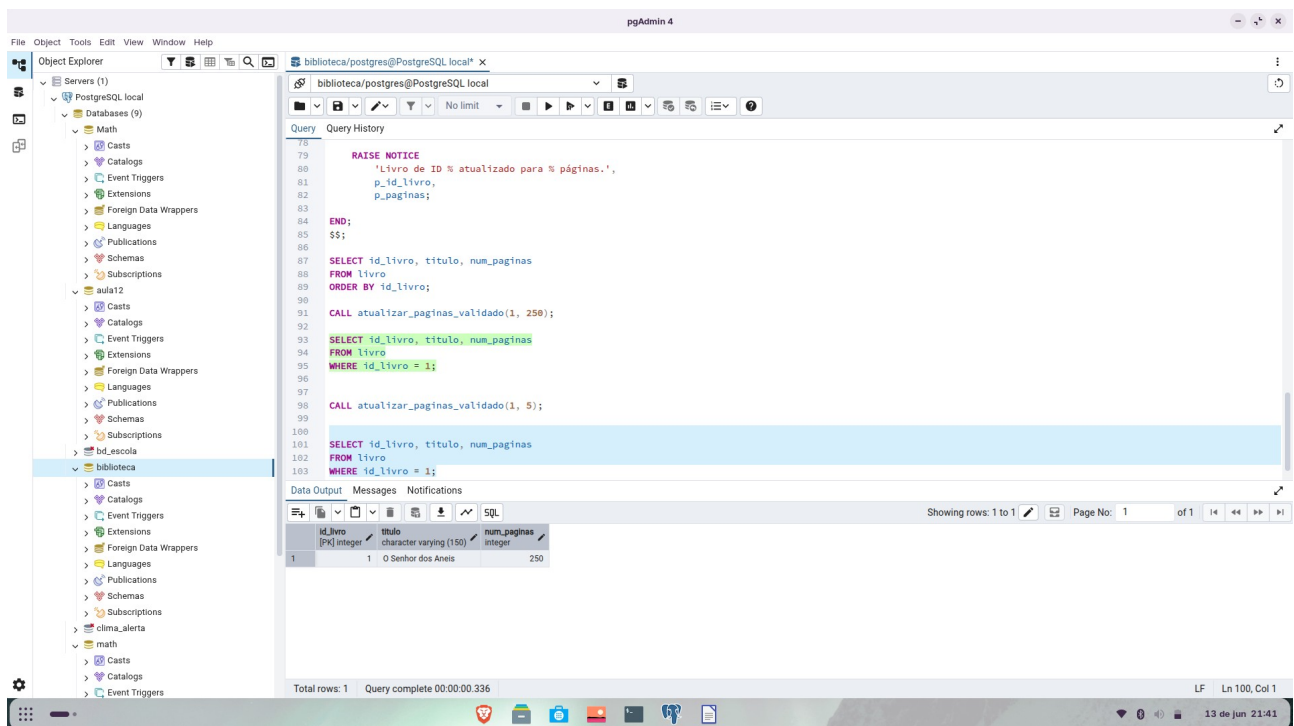
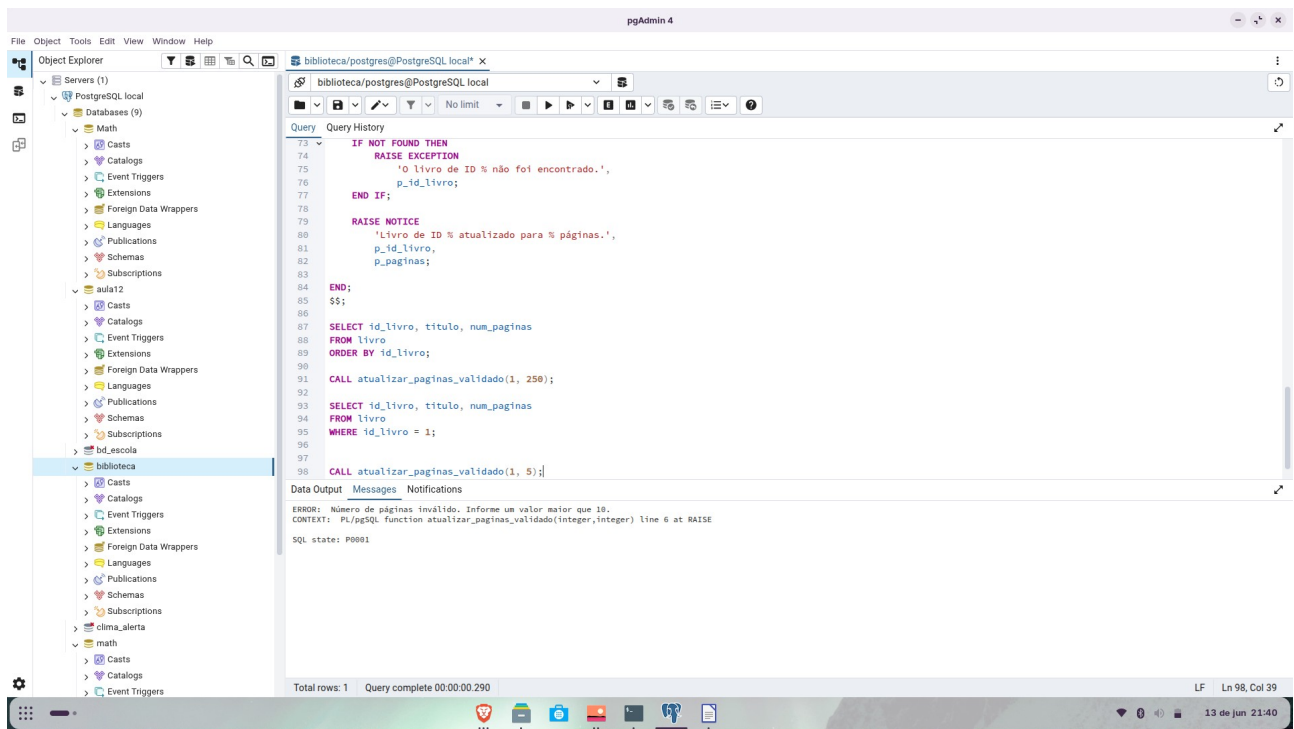
Total rows: 4 Query complete 00:00:00.308

The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, with the 'biblioteca' database selected. The main pane shows a SQL query being executed. The query is a PL/pgSQL function that updates the number of pages for a book and raises an exception if the book is not found. The results pane shows the output of the query, which is a table with 1 row.

```
Query History
SET num_paginas = p_paginas
WHERE id_livro = p_id_livro;
-- Verifica se o livro foi encontrado
IF NOT FOUND THEN
RAISE EXCEPTION
'livro de ID % não foi encontrado.',
p_id_livro;
END IF;
RAISE NOTICE
'livro de ID % atualizado para % páginas.',
p_id_livro,
p_paginas;
END;
$$;
SELECT id_livro, titulo, num_paginas
FROM livro
ORDER BY id_livro;
CALL atualizar_paginas_validado(1, 256);
SELECT id_livro, titulo, num_paginas
FROM livro;
```

id_livro	titulo	num_paginas
1	O Senhor dos Anéis	250

Total rows: 1 Query complete 00:00:00.257



-- EXERCÍCIO 1

CREATE OR REPLACE PROCEDURE inserir_livro_se_autor_existe(p_titulo VARCHAR, p_ano INT, p_id_autor INT, p_id_editora INT, p_paginas INT) LANGUAGE plpgsql AS \$\$ BEGIN

```

-- Verifica se o autor informado existe
IF NOT EXISTS (
    SELECT 1
    FROM autor
    WHERE id_autor = p_id_autor
) THEN
    RAISE EXCEPTION
        'Não foi possível inserir o livro. O autor de ID % não existe.',
        p_id_autor;
END IF;

-- O INSERT só será executado se o autor existir
INSERT INTO livro (
    titulo,
    ano_publicacao,
    id_autor,
    id_editora,
    num_paginas
)
VALUES (
    p_titulo,
    p_ano,
    p_id_autor,
    p_id_editora,
    p_paginas
);

RAISE NOTICE 'Livro "%" inserido com sucesso.', p_titulo;

```

END; \$\$;

-- ===== -- EXERCÍCIO 2

CREATE OR REPLACE PROCEDURE atualizar_paginas_validado(IN p_id_livro INTEGER, IN p_paginas INTEGER) LANGUAGE plpgsql AS \$\$ BEGIN

```

-- Valida a nova quantidade de páginas
IF p_paginas <= 10 THEN
    RAISE EXCEPTION
        'Número de páginas inválido. Informe um valor maior que 10.';
END IF;

-- Atualiza o livro
UPDATE livro
SET num_paginas = p_paginas
WHERE id_livro = p_id_livro;

-- Verifica se o livro foi encontrado
IF NOT FOUND THEN
    RAISE EXCEPTION
        'O livro de ID % não foi encontrado.',
        p_id_livro;
END IF;

RAISE NOTICE
    'Livro de ID % atualizado para % páginas.',
    p_id_livro,
    p_paginas;

```

END; \$\$;

SELECT id_livro, titulo, num_paginas FROM livro ORDER BY id_livro;

```
CALL atualizar_paginas_validado(1, 250);
```

```
SELECT id_livro, titulo, num_paginas FROM livro WHERE id_livro = 1;
```

```
CALL atualizar_paginas_validado(1, 5);
```

```
SELECT id_livro, titulo, num_paginas FROM livro WHERE id_livro = 1;
```